unidb

Engine Architecture & Design Reference

*One embedded Rust engine unifying relational CRUD, vector search, graph edges, and a durable event queue over **one page store, one WAL, one buffer pool, one transaction manager** — so one transaction can touch all four models atomically.*

Document date: 2026-07-13

Covers: M0–M11, Phases 1–6 complete, CRUD Phase A/B, Milestone P (parallel scan), items 11–17, Milestone 18 (introspection), `JOIN … USING`, Milestone 20 (realtime dispatcher), items 21 (observability) & 22 (logs)

Sources: `CLAUDE.md` · `MEMORY.md` · `PROGRESS.md` · `docs/design/engine_design.md` · `docs/positioning.md` · `docs/backlog/roadmap.md`

Descriptive reference — locked decisions live in `CLAUDE.md` §3 and require human sign-off to change.

Contents

1. Executive summary & design thesis

unidb is a single embedded storage/transaction engine written in Rust. It unifies four data models over one shared storage and transaction core:

- **Relational CRUD** — a practical SQL subset over MVCC-versioned heap tables, with joins, aggregates, subqueries, CTEs, a cost-based optimizer, and `EXPLAIN [ANALYZE]`.
- **Vector search** — `VECTOR(n)` columns, a durable on-disk IVF-Flat index, and a `NEAR` operator (pgvector-style Euclidean; cosine metric available).
- **Graph** — edge records with adjacency indexes and a Cypher read subset.
- **Event queue** — a durable event stream with Kafka-style consumer offsets and replay.

The competitive thesis is **eliminating the multi-system dual-write tax**. “Save a row + its embedding + a graph edge + emit an event” is **one WAL append and one commit** here, versus 3–4 network round-trips with no shared transaction across Postgres + a vector store + a graph DB + Kafka. Measured against a real replaced stack at matched flush-to-platter durability, unidb’s one atomic commit is **3.61× faster** (250 vs 69 txns/s) — and the **crash-consistency win is unconditional**: unidb recovers 0 orphans where the four-system stack recovers a torn record.

> **Honest boundary (by charter):** unidb does not try to out-Postgres Postgres on single-model workloads and expects to lose some single-table CRUD benchmarks against a specialized incumbent. Non-goals: distributed consensus (single-primary only), full ANSI SQL, and a cloud control plane. The headline benchmark is the cross-domain transactional workload against the replaced stack.

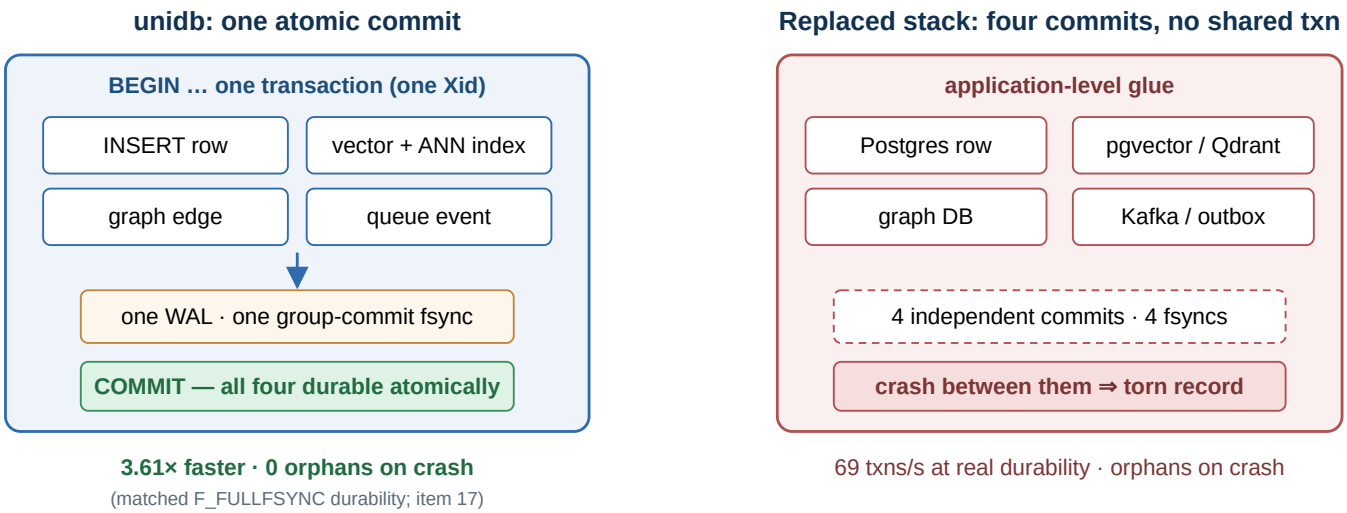

Figure 1 — The moat: one atomic multi-model commit vs the replaced four-system stack (measured, `benches/decompose.rs` Table 4).

2. Architecture overview

2.1 Layer stack

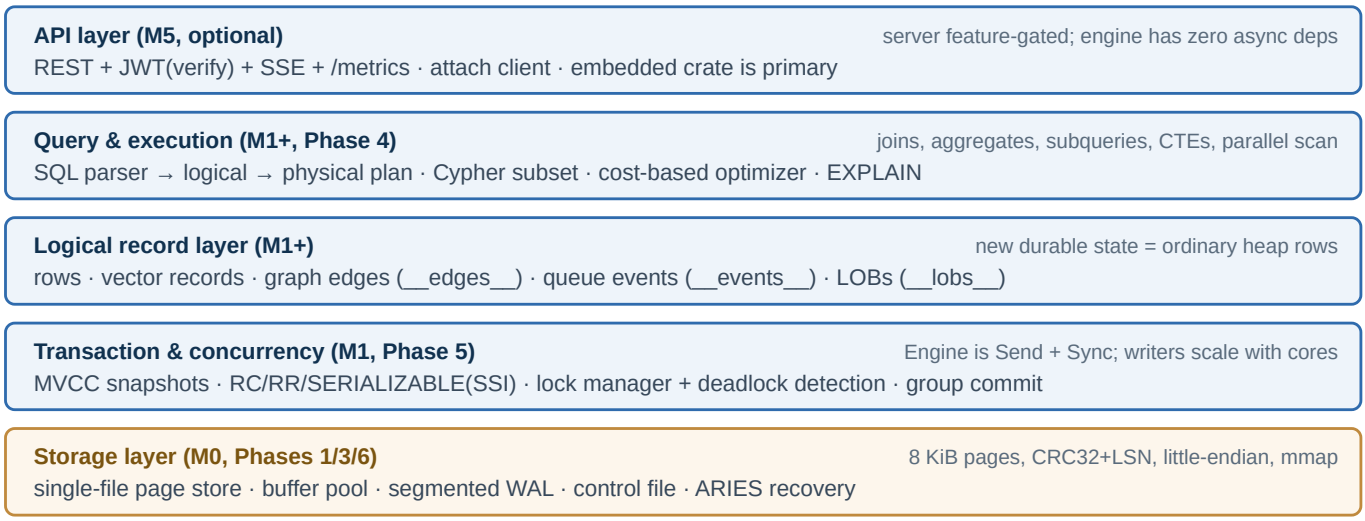

Figure 2 — The five-layer stack. Everything sits on the single shared storage core; that is what makes cross-model atomicity structural rather than promised.

2.2 Module map

Layer	Modules (in `src/`)
Storage core (M0)	`format.rs`, `control.rs`, `mmap.rs` (only `unsafe` module), `page.rs`, `bufferpool.rs`, `wal.rs`, `heap.rs`, `checkpoint.rs`, `recovery.rs`
Transactions (M1; concurrent since Phase 5)	`mvcc.rs`, `txn.rs`, `lockmgr.rs`, `concurrency_hooks.rs`, `query_limits.rs`
Catalog & SQL (M1)	`catalog.rs`, `sql/{parser,logical,executor}.rs`
Query power (Phase 4)	`sql/{query,plan,query_exec,join,aggregate,sort,optimizer,statistics,explain}.rs`
Secondary indexes (durable since Phase 3)	`btree_index.rs` (`DiskBTree`; also backs full-text, edge index, and the durable per-table FSM), `disk_vector.rs` (`DiskIvfIndex`), `fulltext.rs`; retired: `vector.rs` (in-RAM HNSW baseline), `csr_index.rs`
Graph (M3, M7)	`graph/{edges,index,logical,parser,executor}.rs`
Event queue (M4)	`queue/{mod,payload}.rs`

Server (M5, feature-gated)	`server/{engine_handle,error,dto,handlers,router,auth,sse,tls,txn_session,cursor}.rs`, `bin/unidb-server.rs`
Ops & HA (Phase 6)	`replication/`, `backup/`, `authz/`, `audit/`, `autovacuum.rs`
Facade / clients	`lib.rs` (`Engine` — sole entry point), `read_handle.rs`, workspace crate `unidb-attach`, `unidb-embed` CLI

2.3 Engine-wide invariants

- **Synchronous, `Send + Sync`, shared across threads.** Every method takes `&self`; all mutable state is behind interior-mutable latches/locks/atomics. A default build has **zero async dependencies** (no tokio/reqwest/axum) — verified by `cargo tree`. Durability under concurrency uses a leader-elected **group-commit** barrier (`Wal::sync_up_to`) whose fsync runs with the append lock released, so concurrent committers coalesce behind one fsync.
- **Explicit transactions everywhere.** Every operation takes an explicit `Xid` from `Engine::begin` / `begin_with_isolation` and ends with `commit` / `abort`. There is no implicit transaction anywhere in the crate.
- **WAL before page, always (D5).** No code path writes a page before its WAL record; a dirty page is never flushed/evicted while `page.LSN > durable_WAL_LSN`. Tested, not folklore.
- `#![forbid(unsafe_code)]` except the isolated `mmap.rs`; no `unwrap()` / `expect()` outside tests; no `serde` on the page/WAL hot path (hand-rolled little-endian encodings).

2.4 Deployment modes

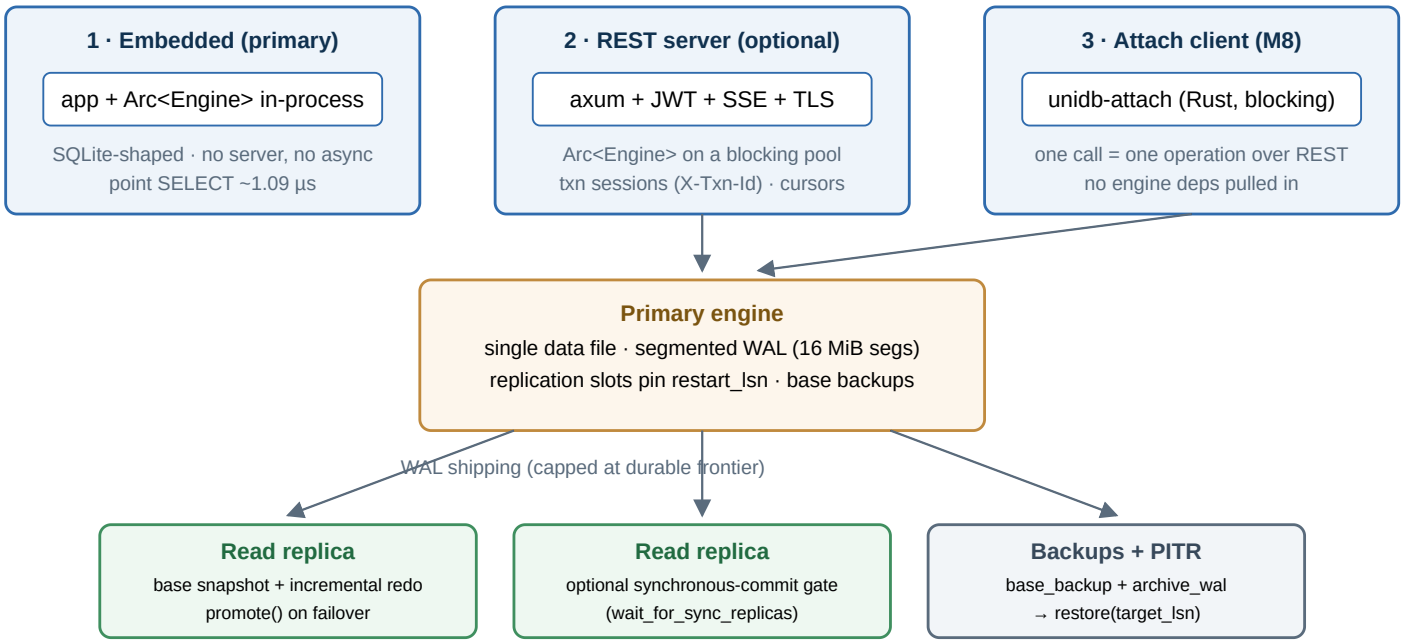

Figure 3 — Deployment modes and the Phase-6 HA topology: embedded (primary mode), optional REST server, attach client, and single-primary + read replicas with PITR.

3. Storage core

3.1 On-disk format (D6, D8, D9)

- **Single data file** of fixed-size pages; the WAL is a separate, segmented directory (`db.wal/seg-*.wal` , Phase 6). Multi-file storage was deliberately rejected (D6): it forces per-file LSN tracking into recovery for benefits not yet needed.
- **8 KiB pages** by default (D8), config-overridable at init only, baked into the control file; immutable once files exist.
- **Little-endian everywhere on disk** (D9). Every page carries a CRC32 checksum and an LSN. Page and WAL records are hand-rolled byte encodings — no `serde` on the hot path.
- **`FORMAT_VERSION` = 5**: v2 tuple-header extension (M1) → v3 `next_xid` control field → v4 `WAL_FPI` torn-page record (P1.a) → v5 durable B-Tree (`WAL_INDEX` record, `PAGE_TYPE_BTREE` pages, `index_root` catalog pointer).
- **Slotted pages**; tuples carry a 24-byte header with `xmin` / `xmax` / `prev_page` / `prev_slot` (reserved in M0 per D4, in active MVCC use since M1).

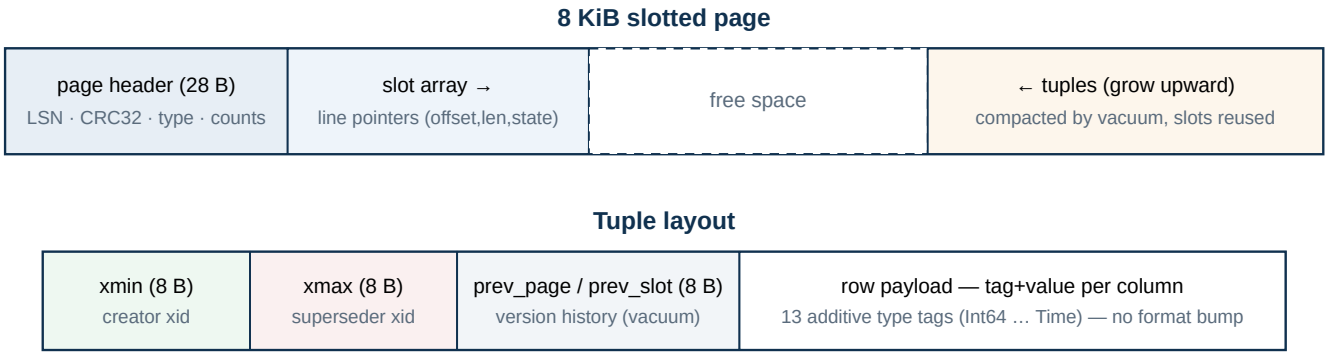

Figure 4 — Page and tuple layout. Every page: CRC32 + LSN; every tuple: MVCC header + hand-rolled little-endian column encoding.

3.2 Control file (D3)

The recovery root — unidb's `pg_control` . 44 bytes: magic, format version, `page_size` , last-checkpoint LSN, WAL tail pointer, `catalog_root` (page id of the current catalog blob), `next_xid` , CRC32. Created at DB init; **recovery always starts here**. Persisting `next_xid` at each checkpoint fixed a real silent-corruption bug: the xid counter used to be recovered by scanning WAL `TXN_BEGIN` records, which checkpoint truncation deleted — a reissued xid could collide with existing `xmin` / `xmax` stamps.

3.3 WAL, mini-transactions & the durability protocol (D1, D2, D5)

- **Steal + no-force, ARIES-style (D1):** the WAL carries both **redo and undo** information per record. Record framing: `u32` LE length prefix + per-record CRC32; a recovery scan stops cleanly at the first corrupt/truncated record.
- **Mini-transactions (D2):** each statement is a WAL-bracketed group of page writes (begin/commit records) that redo/undo treat as one unit. User transactions add `WAL_TXN_BEGIN/COMMIT/ABORT` on the same wire format (the `mini_txn_id` u64 slot doubles as the `Xid`). Additive record kinds: `WAL_VACUUM` (redo-only reclamation), `WAL_FPI` (full page image), `WAL_INDEX` (redo-only index node image).
- **Group-committed force-log-at-commit (default):** statement mini-txns inside an open transaction append *without* a per-statement fsync; `Engine::commit` forces the commit record durable via `Wal::sync_up_to` — **one group-**

coalesced fsync per transaction. The leader's `sync_all` runs with the append lock released, so concurrent committers coalesce behind one fsync. No commit is acknowledged before its commit LSN is synced; a `synchronous_commit=off`-style ack-before-flush is deliberately not offered. Standalone durability-claiming operations (checkpoint, vacuum, index create, slot metadata, backups) self-sync.

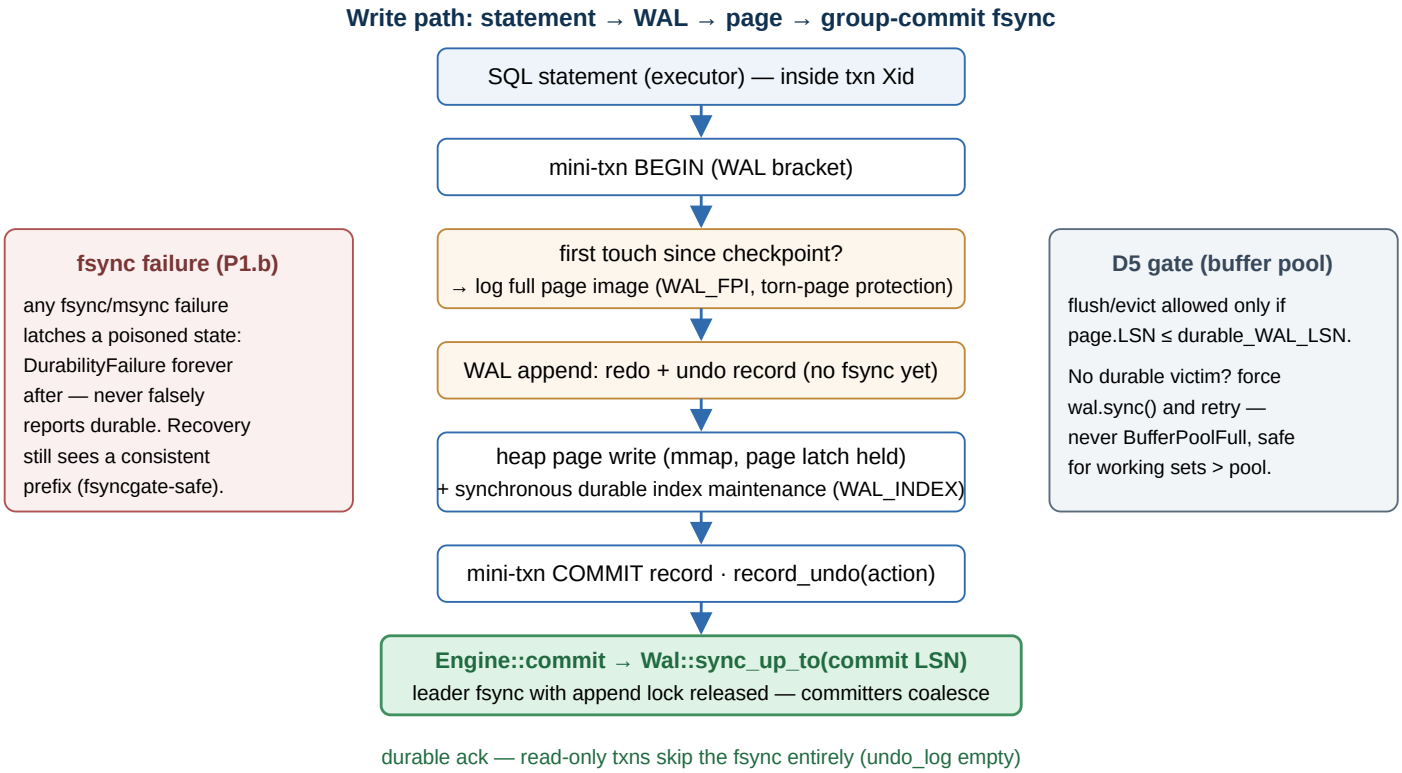

Figure 5 — The write path. WAL-before-page is enforced at every mutation; durability is one group-coalesced fsync per transaction (ARIES force-log-at-commit).

3.4 Torn-page protection & fsync hardening (Phase 1)

- **Full-page writes (P1.a).** An 8 KiB page write is not atomic; a crash mid-write leaves a half-old/half-new page that CRC detects but cannot repair — the #1 silent data-loss hole. On the first modification of a page after each checkpoint, the buffer pool logs the entire clean page image as a redo-only `WAL_FPI` record; recovery replays it as the clean base and re-applies later incremental redos on top. Correct with one image per page per checkpoint interval because D5 guarantees any torn on-disk page belongs to a committed mini-txn whose FPI is in the redo set.
- **fsync-failure poisoning (P1.b, "fsyncgate").** A failed fsync may leave the OS having dropped dirty data while clearing its dirty bit, so a retry can falsely succeed. `Wal` and `BufferPool` latch a poisoned state on any durability-primitive failure and refuse all later durability claims; the session must restart, and recovery still sees a consistent prefix.

3.5 Buffer pool

- **Configurable capacity**: 4096 frames default (32 MiB), via `UNIDB_BUFFER_POOL_PAGES` or `open_with_pool_capacity`. Per-page shared/exclusive latches (Phase 5); clock eviction; dirty tracking; D5 enforcement in exactly two places (`flush_page()` and `find_victim()`).
- **mmap-as-storage:** `Frame` holds eviction metadata only; `write_page` writes into the mmap and `read_page` returns an owned copy under the read lock — which is why parallel scan workers can read the shared mmap with no pool-staleness hazard (verified during Milestone P, not assumed).

- **Proper ARIES steal:** `find_victim` writes back and evicts a dirty page once its LSN is durable; if every dirty frame leads the durable WAL (group-commit deferral), the write path forces one `wal.sync()` and retries — "force the log before stealing the page", never `BufferPoolFull`.
- **Chunked file growth:** the file grows in 4 MiB chunks; the mmap is re-created only when a new page crosses the mapped boundary (was a whole-file remap per `alloc_page` — O(N²)). Benchmarked flat at ~1M `alloc_page`/s to 100k pages.

3.6 Free-space map (durable, per table)

A catalog table's page directory + free-space map is a per-table durable `DiskBTree` keyed `page_id → free_bytes` (`TableDef.fsm_meta`). `Heap::open` is O(1); the SQL insert path appends at the durable tail via `DiskBTree::max_entry` (O(log n), no per-statement rebuild); vacuum reclamation is durable. This replaced the in-catalog page-list blob whose single-page cap produced `HeapFull` at ~145k rows (~876 pages) — marginal SQL-insert cost went from rising-then-fatal (65→108→173 µs/row) to flat ~17–28 µs/row past 2,000 pages.

3.7 Checkpoint & recovery

`checkpoint::run`: flush all dirty pages → reset FPI tracking → write a checkpoint WAL record → persist the control file (checkpoint LSN, WAL tail, `catalog_root`, `next_xid`) → truncate consumed WAL. **Auto-checkpoint (P1.e)** fires on a time trigger (default 60 s) or a WAL-size trigger (default 64 MiB), but only at a quiescent point (no active transactions) — a permanently open transaction blocks it (documented footgun, as in Postgres). Since Phase 6 the WAL is **segmented** (16 MiB segments); truncation deletes whole consumed segments, and the truncation floor is `min(checkpoint_lsn, min slot restart_lsn)` so replication slots pin what replicas still need.

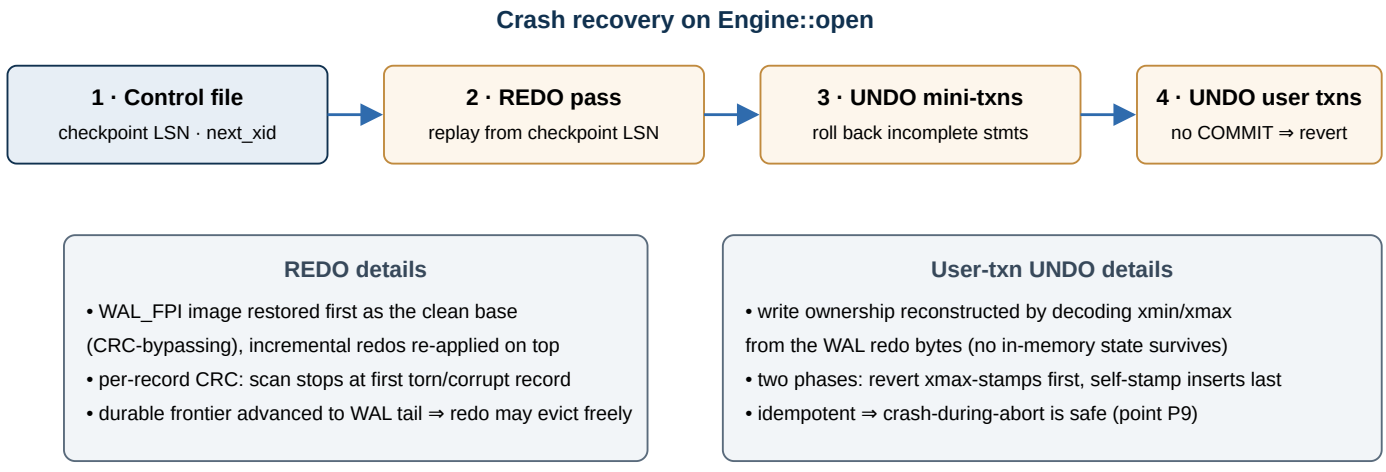

Figure 6 — ARIES-style recovery: control file → redo (FPI-based) → undo incomplete mini-transactions → undo incomplete user transactions.

3.8 Crash-injection harness (D7)

`tests/crash/` kills the process at defined points, reopens, and asserts the recovered state equals the expected committed set — **31 tests** as of item 17. Points cover: post-WAL/pre-flush, mid-checkpoint, post-mutation/pre-commit, during WAL truncation, post-commit-fsync (P1–P5); user-txn boundaries and mid-undo (P6/P7/P9); mid-vacuum (P10); torn 8 KiB page recovered from `WAL_FPI` (P11); fsync-failure poison (P12); total data-file loss reconstructed from WAL alone (P13); durable full-text/edge/LOB/ vector indexes (P14–P17, vector with recall intact); segmented-WAL multi-segment recovery + truncation (P18); backup + PITR restore after primary loss (P19); group-commit deferral points (Pa–Pd); durable FSM (P27/P28); and the replaced-stack crash-consistency proofs (`item16_*`). A property test runs random `BEGIN/INSERT/COMMIT/ROLLBACK` sequences with random crash points under **both** durability policies. The harness must grow whenever a new durability mechanism lands.

4. Transactions, MVCC & concurrency

4.1 Version model & visibility

unidb uses **insert-new-version MVCC**: UPDATE always creates a new `RowId` and stamps the old version's `xmax` — never in-place. `prev_page` / `prev_slot` record version history for recovery/vacuum bookkeeping only; **readers never walk the chain** — `Heap::get` is a single direct visibility check. Visibility (`mvcc.rs`) is pure snapshot logic: a `Snapshot` (active-xid set + range) and `is_visible(tuple, snapshot)` . There is deliberately **no "aborted" state** in visibility — abort therefore requires physical undo, and (item-16 correction) that undo must run while the aborting xid is still in the active set.

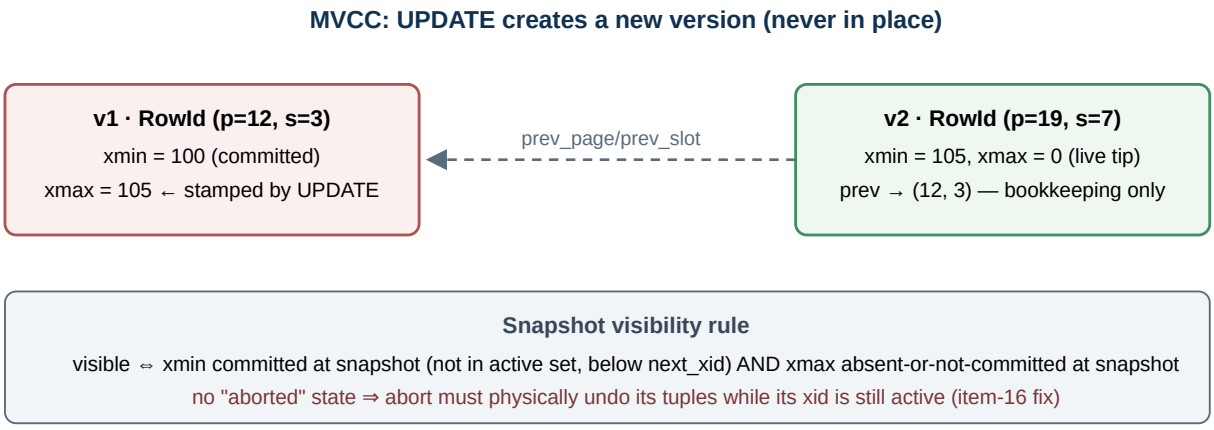

Figure 7 — Insert-new-version MVCC. A snapshot before xid 105 commits sees v1; after, v2. Vacuum reclaims dead versions behind a reader-aware horizon.

4.2 Isolation levels (D10, D12; SSI in P1.d)

Level	Snapshot lifetime	Conflict behaviour
READ COMMITTED (default)	per statement	Each statement re-reads the latest committed tip via its fresh snapshot (EvalPlanQual is inherent to a re-scanning executor — no spurious abort). A conflict against a still-active writer is rejected (`WriteConflict` , no-wait by design).
REPEATABLE READ (= snapshot isolation)	per transaction	Write-write conflict surfaces as `SerializationFailure` — the caller retries.
SERIALIZABLE (SSI)	per transaction	RR's snapshot + Cahill-style rw-antidependency (pivot) detection over per-txn read/write sets; a pivot aborts at commit with `SerializationFailure` , preventing write-skew. Reduced form: row-granularity (no predicate locks ⇒ no phantom protection), occasionally over-conservative.

The `on_read()` / `on_write()` seam (D11) exists in every scan/lookup path (`concurrency_hooks.rs`), kept no-op — SSI tracks at the executor; the seam remains available for finer-grained tracking later.

4.3 Locking & deadlock detection

- Write-write locks only (no read locks under MVCC), keyed by `RecordId::row(page_id, slot)` packed into a u64. Because `PageId` comes from one global allocator, lock keys are globally unique across all tables — graph edges needed **zero** new locking code.

- Locks are held until commit/abort, so there is no window between write and commit for another writer to slip through (no commit-time recheck needed).
- Phase 5 made the lock manager real: S/X modes, blocking `Condvar` wait queues, and wait-for-graph deadlock detection.
- Per-query **timeouts, cancellation, and `work_mem`** via `Engine::execute_sql_with_limits` (P5.f); parallel scan workers check the deadline every few pages.

4.4 Abort = physical self-neutralization

Abort self-stamps `xmax := own xmin` on every tuple the transaction inserted and reverts every xmax-stamp it applied, driven by an in-memory `Vec<UndoAction>`. `Heap` never records undo itself — every call site pairs the heap call with an explicit `record_undo(...)`, which is what let edges/events/LOBs inherit correct abort behavior with zero new abort-path code (guarded by per-feature abort-visibility tests, since a forgotten `record_undo` is a silent visibility bug).

> **The item-16 lesson (2026-07-12), a real concurrency bug:** `abort()` originally removed the xid from the active set *before* physically undoing its writes. Because visibility has no "aborted" state, a concurrent snapshot briefly classified the aborting transaction's doomed writes as committed — wrong reads, and a concurrent writer could chain onto the unlocked new version, leaving **two live versions of one row (or none)** after undo. Fix: keep the xid active (and its row locks held) through the whole physical undo. Proven by a deterministic pin-the-midpoint test; took the 28-cell concurrency matrix from 17 PASS/11 FAIL to **28/28**.

4.5 Concurrent SQL writes — latch-coupled B-Tree descent

Raw-CRUD writers have scaled since Phase 5 (group commit; **3.68× at 8 writers**). SQL DML originally serialized on the catalog write lock; the index-write-concurrency milestone (default-ON since item 11, 2026-07-13) lets catalog-non-mutating DML take a *shared* catalog lock, with `DiskBTree` maintenance made race-safe by **latch-coupled ("crabbing") descent with safe-node early release**: latch child before releasing parent; drop all ancestor + meta latches at the first node that cannot split; splits propagate only through the still-latched suffix; latches are taken strictly root→leaf so inserts cannot deadlock. Reads stay latch-free (owned page copies + right-linked leaves + MVCC re-validation). Validated by a structural validator, deterministic split-contention tests, a `loom` model of the latch ordering, and the 28-cell concurrency matrix (28/28 at 10 repeats, toggle on and off). Measured: indexed 8-writer INSERT **811 → 1016 commits/s (+25%)**; revert is one env var (`UNIDB_CONCURRENT_SQL_WRITES=0`).

5. SQL layer

5.1 Catalog & types

`catalog.rs` persists `TableDef` / `ColumnDef` as a single `serde_json` blob rewritten to a fresh page on every change, pointed at by `control.catalog_root` (schema is control-plane data, not the hot path D9 protects). Column types: `Int64`, `Text`, `Bool`, `Json`, `Vector(n)`, `Decimal(p,s)`, `Timestamp`, `Float`, `Uuid`, `Bytea`, `Date`, `Time` — encoded as additive tag+value pairs (no format bump for new tags). The catalog is **not MVCC-versioned**: DDL takes effect immediately and globally (request-level DDL rollback exists; crash-safe transactional DDL is tracked debt). `SERIAL` is a durable per-table counter; prepared statements + `$n` bind parameters close the SQL-injection surface. **Introspection (Milestone 18, 2026-07-13):** the catalog is queryable via `information_schema.{tables, columns, table_constraints, key_column_usage, …}` / `unidb_catalog.*` virtual tables, resolved at plan time and identical from embed, attach, and server — the documented access-and-introspection contract applications build on (see `docs/engine_access_guide.md`).

5.2 Query pipeline

```
SQL text ──sqlparser (GenericDialect)──▶ LogicalPlan
  ├─ trivial single-table SELECT ──▶ original fast path (feeds the concurrent-read path)
  │     ├─ B-Tree-indexable predicate? ──▶ try_exec_select_btree (index → resolve → refilter)
  │     ├─ NEAR(col, [..], k)? ──▶ DiskIvfIndex probe → exact re-rank → MVCC/RLS recheck
  │     └─ full scan ──▶ deform_row (decode only needed columns) · parallel workers if eligible
  └─ anything richer ──▶ LogicalPlan::Query(QuerySpec) ──▶ cost-based optimizer
        (ANALYZE stats · Selinger left-deep DP join order · index-vs-scan)
        ──▶ physical operator tree: hash join (Grace spill) / sort-merge /
            index-nested-loop · aggregate · external merge-sort · subqueries/CTEs
        ──▶ EXPLAIN [ANALYZE] renders the tree with estimated vs actual rows
```

Executed row-at-a-time. Row-level security is a planner rewrite: `apply_rls` ANDs the stored policy predicate onto the statement's predicate — one function, composing for free with `NEAR` and index-assisted paths because every path filters through the same `predicate_matches`.

5.3 Query power (Phase 4) — what ships

Area	Supported	Out of scope (documented)
Joins	INNER/LEFT/RIGHT/CROSS; `USING (cols)` (shipped 2026-07-13, planner desugar + shared-column merge); hash (with Grace spill-to-disk), sort-merge, index-nested-loop over the durable B-Tree	FULL OUTER, NATURAL
Aggregation & sort	COUNT/SUM/AVG/MIN/MAX, GROUP BY/HAVING, DISTINCT, ORDER BY (external merge-sort spill), LIMIT/OFFSET	window functions
Subqueries & CTEs	scalar/IN/EXISTS (correlated + uncorrelated), IN (list), non-recursive WITH	recursive CTEs
Planning	ANALYZE (durable per-table stats: row counts, distinct counts, equi-depth histograms), cost-based join order (Selinger DP ≤ 10 relations), index-vs-scan choice, EXPLAIN [ANALYZE]	columnar / vectorized execution (parked)
DDL & schema	CREATE/ALTER/DROP/TRUNCATE, tombstone DROP COLUMN, request-level DDL rollback, SERIAL, prepared statements + $n params	transactional DDL through crash recovery

Correctness for joins/aggregates/subqueries is checked differentially against SQLite on shared data.

5.4 Constraints & security

- **Constraints (M11):** PRIMARY KEY, FOREIGN KEY/REFERENCES, UNIQUE, NOT NULL, CHECK, DEFAULT — enforced on INSERT/UPDATE. UNIQUE is a synchronous heap scan under the writer's snapshot (an index can be a stale hint — the M7 lesson); FK enforces referenced-table existence.
- **RLS:** per-table policy predicate; settable from Rust (`set_rls_policy`) or over REST as a SQL predicate string parsed by the ordinary parser (`set_rls_policy_sql`) — no `Expr` wire format. No `CREATE POLICY` statement yet.
- **Users/roles/GRANT (P6.e):** `authz::RoleStore` with transitive role membership + per-table privileges; `execute_sql_as(user, ..)` enforces them; the embedded API is the implicit superuser; the server maps the JWT `sub` claim to a user.

6. Secondary indexing framework

Since Phase 3, **every** secondary index is durable, synchronous, WAL-logged, and crash-recovered — **never rebuilt on open** (`Engine::open` is O(1) regardless of data size; the old async index worker is retired). All index-assisted reads use the same **resolve-then-refilter template**: index → candidate `RowId`s → `Heap::get` under the caller's snapshot → full `predicate_matches` (remaining AND terms + RLS apply identically to a full scan).

6.1 Durable B-Tree (`DiskBTree`)

- On-disk B+tree whose nodes are pages in the shared page store — standard 28-byte header, so buffer-pool CRC + D5 discipline apply unchanged; a body tag distinguishes meta/internal/leaf; leaves are right-linked for range + duplicate walks.
- Every insert/remove is **one WAL mini-transaction** logging full node-page images (`WAL_INDEX`, redo-only): recovery redoes all pages of a committed index mini-txn or none. A stable **meta page** (id in `ColumnDef.index_root`) points at the root, so a root split never rewrites the catalog.
- **No undo** — entries are MVCC-validated hints; the one dangerous case (a committed visible row lacking its entry) is prevented by ordering: the index mini-txn fsyncs before the user txn's `WAL_TXN_COMMIT`.
- Also backs: full-text postings, the edge-adjacency index, IVF cell posting lists, and each table's durable FSM. Concurrent-writer safety via crabbing (§4.5). v1 leaves underfull nodes un-merged (tree only grows).

6.2 Durable vector index — on-disk IVF-Flat (`DiskIvfIndex`)

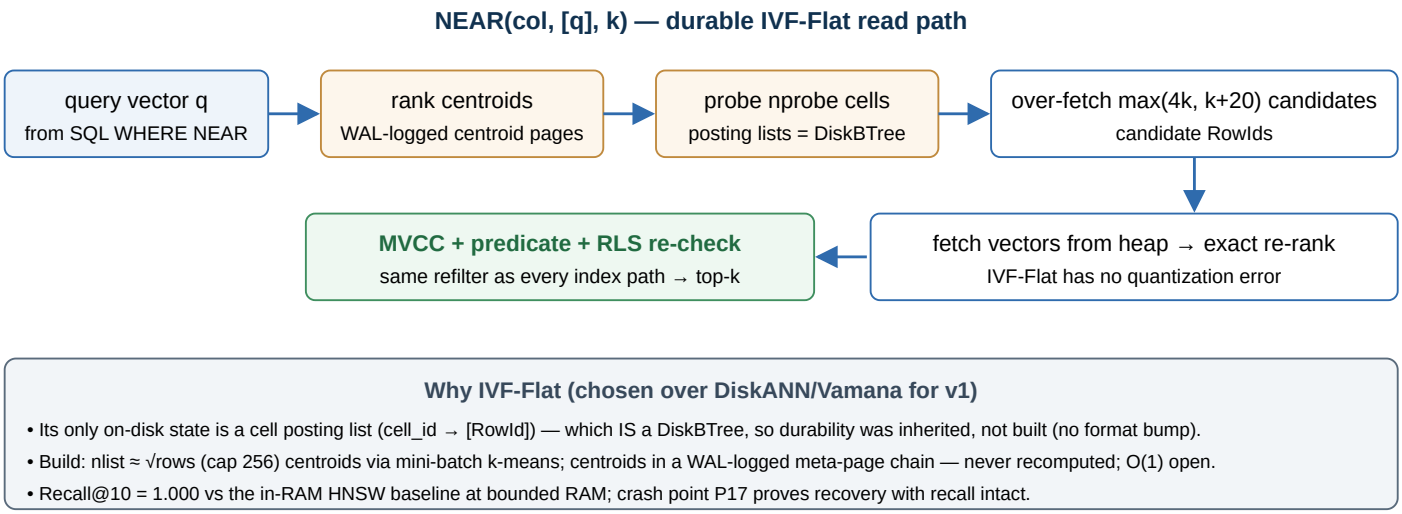

Figure 8 — The `NEAR` operator over the durable IVF-Flat index. The `USING HNSW` keyword now denotes this index; the in-RAM HNSW is retained only as a benchmark baseline.

6.3 Full-text & edge indexes

- **Full-text (inverted):** a durable `DiskBTree` keyed on tokens, one `(token, RowId)` entry per token; read path `Engine::search_fulltext` — tokenize → intersect per-token posting lists (AND-only) → MVCC-resolve. Raw search ~14.2 µs.
- **Edge adjacency:** a durable `DiskBTree` over `__edges__.from_id`, maintained synchronously by `create_edge` / `delete_edge`; meta page cached on the engine. No abort-time cleanup needed — stale entries are permanently safe (snapshot re-validation).

6.4 The correctness pattern: hints + MVCC re-validation

> **Rule 1:** secondary indexes are non-transactional derived state whose entries are **hints, re-validated against MVCC** — stale/phantom entries are space leaks, never wrong answers.
> **Rule 2:** new durable state is always ordinary heap rows (`__edges__` , `__events__` , `__consumers__` , `__lobs__`) — inheriting MVCC, locking, WAL, recovery, and abort handling with zero new mechanism (why M3–M5 added no new crash points).
> **Rule 3:** completeness-sensitive readers must gate on how current an index can ever *promise* to be, not just "ready" — the M7 CSR tier-selection bug (a transaction's own just-created edge invisible to its own read) is the canonical counterexample; CSR was reverted and later retired.

7. Graph layer

- **Edges are ordinary rows** `(from_id, to_id, edge_type, props)` in a synthetic `__edges__` table. Needed **zero** new storage/locking code — per-edge locking, first-committer-wins, and release-on-commit were inherited and then proven by `tests/graph_locking.rs`. Nodes are opaque `i64` IDs.
- **Read path:** durable edge index → candidate RowIds → `resolve_candidates_batched` groups candidates by page so a hot hub costs one page fetch per ~128 edges instead of one per edge — measured **~9.3–9.7×** over naive resolution, closing the gap with Postgres to ~1×–1.6× (94.3 µs vs 98 µs at 1k edges).
- **Cypher subset:** `MATCH (a)-[:TYPE]->(b) WHERE … RETURN …` — exactly one fixed-length directed hop, read-only; the `:TYPE` filter and `WHERE` are AND'd into one predicate applied through the SQL layer's own `predicate_matches`, so the index fast path and full-scan fallback filter identically. Mutations are Rust-API `create_edge` / `delete_edge`.

8. Event queue

Why not tail the WAL: checkpointing truncates the WAL unconditionally (making retention depend on external readers would be D5-adjacent bad news), and WAL records carry no table identifier. Instead, **events are ordinary rows**: `send_event_capture` copies each triggering row into a durable `__events__` table *inline, at write time, under the writing transaction's own xid*. Consequences — all structural:

- WAL truncation cannot care about consumer lag (proven: checkpoint five times under a never-acking consumer, still poll every event).
- Events get MVCC, durability, abort semantics (an aborted transaction's events vanish with it), and plain SQL queryability for free.
- Consumer offsets live in `__consumers__`, updated by `ack_events` under the caller's transaction — an aborted ack does not advance the offset. `poll_events` / `ack_events` are Kafka-style manual-commit; `vacuum_events()` reclaims rows acknowledged by **every** registered consumer.

Measured: `poll_events` 20.8–983.7 µs (100–5,000 rows) vs a Postgres SKIP-LOCKED cycle at ~2.6–3.1 ms — though unidb's cost grows linearly with table size (no `seq` index yet; `vacuum_events` is the bounding lever). The server exposes the queue as SSE (`GET /events/subscribe`), currently server-polls-then-pushes.

8.1 Realtime dispatcher (Milestone 20, 2026-07-13)

The event stream is now consumable downstream *without teaching the engine any application shape* (the Milestone-18 boundary holds — the engine emits raw row-level facts; all delivery semantics live outside it). Three parts:

- **SSE resume framing (E1):** `GET /events/subscribe` gained an *ephemeral live-tail* mode — a per-connection cursor seeded from the standard SSE `Last-Event-ID` reconnect header (else `?from_seq=`), plus an optional `?table=` filter; each frame carries `id: <seq>`. Durable-consumer mode is unchanged. Backed by **one new read-only engine method** `poll_events_after(after_seq, limit)` — it truncates *after* filtering so a cursor beyond `limit` never drops the tail. No storage/format/crash surface (harness stays 31).
- **`unidb-dispatch` crate (E2):** a new workspace crate that embeds `Arc<Engine>` and fans the stream out, keeping tokio/reqwest **out of the `unidb` crate entirely** (the "engine stays sync" invariant is literally true). Each cycle: poll from a durable offset → per-subscription table/op filter + column projection → *then* ack. Sinks: `WebhookSink` (exponential-backoff retry → **dead-letter table dogfooded back into unidb**), `RoomSink` (broadcast rooms — the primitive a studio WS/SSE layer subscribes to), `CollectingSink`.
- **Delivery semantics (E3), proven not promised:** I/U/D consumed at-least-once in offset order; **zero loss across an engine crash** (commit 5, ack 3, drop+reopen → restart delivers only {4,5}, union = {1..5}); at-least-once redelivery of an event whose deliver-then-ack straddled a crash; a poison webhook retries 3× then dead-letters while the offset still advances (a poison event cannot wedge the stream). A falling-behind dispatcher pins the vacuum horizon and reports `backlogged` rather than silently dropping.

Honest caveat (§0.6): the dispatcher inherits M4's `poll_events` cost model — no predicate pushdown / no `seq` index — so draining N events is ≈ O(N²/limit) poll work; fast through N≈4k (measured ~95k–120k ev/s drain), but a large backlog needs the engine-side `seq` index (tracked M4 debt). This is the shipped realization of the §14.2 "push realtime" track.

9. Server, attach client & APIs

9.1 REST server (M5 + item 12)

Feature-gated (`--features server`); the default build has zero tokio/axum. Handlers never run `Engine` calls on the async runtime: one shared `Arc<Engine>` , each blocking call on a tokio blocking-pool thread — many requests execute concurrently through the engine's internal latches, with group commit coalescing their fsyncs.

Surface	Routes / behaviour
SQL	`POST /sql` — atomic multi-statement transactions, optional one-shot `isolation` , `$n` params, optional cursor mode (`GET /sql/cursor/{id}` , principal-bound, idle-expiring)
Transactions	`POST /txn/begin` → `X-Txn-Id` on later requests (no auto-commit) → `POST /txn/{id}/commit|rollback` . Per-session busy try-lock (409 `TXN_BUSY`), JWT-principal binding (403), idle reaper auto-aborts abandoned sessions (vacuum horizon un-pinned)
Data / graph / vector	row CRUD (`/rows…` , atomic `POST /rows/batch`), `POST /cypher` , `/edges…` , `POST /indexes` + status
Events	`POST /tables/{t}/events` , SSE `GET /events/subscribe` (+ ephemeral live-tail resume via `Last-Event-ID` / `?from_seq=` / `?table=` , M20), `POST /events/ack` , `POST /events/vacuum`
Admin / security	verify-only JWT (HS256), users/roles via JWT `sub` , superuser-gated RLS (`PUT /tables/{t}/rls`) + `/admin/flush` , `POST /checkpoint` , native TLS (rustls)
Observability (enriched item 21/22, 2026-07-13)	`GET /metrics` (Prometheus), `GET /stats` (`pg_stat_*` -style, now with per-statement-kind + fsync + buffer-pool + lock-wait + horizon-age + parallel-worker + session metrics), `GET /logs` (superuser-gated, bounded cursor-paged JSON-log tail), slow-query log, audit log
Replication	slot + shipping routes (P6.b)

9.1.1 Observability metrics (item 21) & logs surface (item 22)

- **Lock-free chokepoint metrics (item 21):** per-statement-kind latency histograms, WAL-fsync latency + count (`commits / wal_fsyncs` reads out group-commit amortization), buffer-pool hit/miss/eviction, lock-wait count/duration + deadlock counter, an **oldest-snapshot / vacuum-horizon-age gauge** (the alertable item-16 postmortem metric), per-table heap page counts, parallel-worker utilization vs the global budget, and server session gauges. Capture is plain `AtomicU64` / a fixed-bucket `AtomicHistogram` (`src/metrics.rs`) — three `Relaxed` adds, **no mutex on the commit or scan path**; measured overhead is within ±1% at scale. Surfaced only through the existing `stats()` / `/stats` / `/metrics` boundary — no new endpoint (the M18 rule).
- **Structured logs + correlation (item 22):** server logs emit **JSON lines** (stdout + rolling `unidb.log.YYYY-MM-DD` files), a per-request `request_id` (+ `txn_id`) joins one request's lines across the app log, the slow-query log, and `audit.log` (threaded through `spawn_blocking` into the engine thread-local), and `GET /logs` is a **bounded**, cursor-paged reverse read of the JSON files (page cap 500, scan budget 50k lines, 64 KiB block reads) so a multi-GB log directory can neither OOM nor stall the server. The default embedded build gains only a tiny `std` -only `request_id` module — no new dependency, engine stays sync.

9.2 Attach client (M8)

`unidb-attach` : one-shot, `Engine` -like calls over the REST surface for processes not embedding the engine — blocking `reqwest` , no tokio, no background thread. `AttachError` mirrors the server's documented error codes 1:1 (not storage-internal `DbError`). Benchmarked at raw-reqwest parity; HTTP itself is the cost (an order of magnitude over embedded — the expected finding).

10. Operations & high availability (Phase 6)

- **Segmented WAL (P6.a):** `db.wal/` holds fixed-size 16 MiB segments with base-LSN headers; seal + rotate on the boundary; recovery scans segments in LSN order; truncation deletes whole consumed segments (D6 evolved with sign-off).
- **Replication slots + WAL shipping (P6.b):** a persisted `SlotRegistry` pins `restart_lsn`; the truncation floor is `min(checkpoint_lsn, min slot restart_lsn)`. Shipping is capped at the **durable frontier**, so a replica can never apply a commit the primary had not made durable — a replica is always a prefix of the primary's durable state.
- **Read replicas + failover (P6.c):** a replica seeds from a base snapshot and applies shipped WAL incrementally via the crash-recovery redo path; `promote()` opens it read-write; optional `wait_for_sync_replicas` synchronous-commit gate. (Documented limit: pages first allocated after the base need a re-base to be FPI-covered.)
- **Backups + PITR (P6.d):** `base_backup` (checkpoint + copy) + `archive_wal`; `backup::restore(base, archive, dest, target_lsn)` — PITR **by LSN** (time-based needs commit timestamps — a filed follow-up). Measured: base backup 7 ms, restore 72 ms, PITR 43 ms, failover 26 ms at 5k rows.
- **Autovacuum (A1–A4):** a background `std::thread` launcher (holding `Weak<Engine>`) wakes every naptime and runs `Engine::vacuum` when `dead > threshold + scale_factor · live` (Postgres-shape policy). Vacuum computes a reader-aware horizon (live transactions + live `ReadHandle` readers + replication slots), marks reclaimable versions via redo-only `WAL_VACUUM`, **scrubs every secondary index before any slot becomes reusable** (the aliasing gate), then compacts pages. Under 200×30 churn: heap bounded at ~35 pages vs ~82 unvacuumed; point reads restored 35 → 5.85 µs.
- **Security & observability:** users/roles/GRANT, verify-only JWT → per-user privileges, native TLS, append-only audit log, ops runbook (`docs/ops_runbook.md`). Observability was **enriched 2026-07-13** (items 21/22, §9.1.1): `Engine::stats()` now carries lock-free per-chokepoint metrics (statement/fsync/buffer-pool/lock-wait/ horizon-age/parallel-worker/session), and the server adds JSON structured logs with `request_id` correlation across app/slow-query/audit logs plus a bounded `GET /logs` tail (CloudWatch/Datadog/Loki shipping contract). Encryption-at-rest is deferred (D9-sign-off-gated; it fights the mmap page store).

11. Performance engineering (measured, not aspirational)

All numbers from `PROGRESS.md` / `scripts/report.sh` (release builds, Apple Silicon, real flush-to-platter fsync unless noted). Baselines per charter: SQLite for single-model embedded comparisons; Postgres (+pgvector, matched durability) from M2 on; the four-system replaced stack for the headline.

11.1 The one bottleneck that explained almost everything — and its fix

Through M8, **every statement paid its own WAL fsync (~3 ms on the reference hardware)**, putting single-statement writes ~30× behind SQLite and making a 100-row transaction cost ~3.45 ms/row. The fix landed in two steps: group commit on the server (2026-07-08), then **group-committed force-log-at-commit as the engine-wide default** (2026-07-09) — statement mini-txns inside a transaction defer their fsync; `Engine::commit` issues one coalesced fsync. This is ARIES force-log-at-commit and *fulfills* D1 (per-statement fsync was an over-fulfillment inherited from M0). Results: the full multi-model commit dropped **~33.1 → ~4.3 ms (~7.7×)**; server `POST /sql` went from flat ~135–158 ops/s to **~242 / ~756 / ~4,780 ops/s** at 1/10/50 clients; read-only transactions skip the commit fsync entirely (point SELECT 3.05 ms → **~1.09 µs**).

11.2 Performance improvements ledger

Improvement (when)	Mechanism	Measured effect
Group commit + commit-time fsync (M9 → default 2026-07-09)	defer statement fsyncs; leader-elected `sync_up_to`; fsync outside the append lock	multi-model commit 33.1→4.3 ms (~7.7×); server 135→4,780 ops/s @50 clients; W0 at SQLite parity
Read-only fsync skip (M9)	`commit()` skips WAL when `undo_log` empty	point SELECT 3.05 ms → ~1.09 µs
Concurrent writers (Phase 5)	`Send+Sync` engine, per-page latches, real lock manager, worker pool	raw-CRUD commits scale 3.68× at 8 writers (325→1,197/s)
Concurrent SQL writes (item 11, default-ON)	shared catalog lock + crabbing B-Tree descent	indexed 8-writer INSERT 811→1,016 commits/s (+25%); matrix 28/28
Batch-latch graph resolution (M3)	group candidate RowIds by page (~128 edges/page)	~9.3–9.7× over naive; 94.3 µs vs PG 98 µs at 1k-edge hub
Buffer pool & growth (P1.c, M9)	chunked 4 MiB growth (was O(N²) remap); 4,096-frame pool; ARIES steal on eviction	flat ~1M alloc_page/s; heaps to 300k+ rows flat; `BufferPoolFull` eliminated
Durable FSM (2026-07-10)	per-table `DiskBTree` page directory/FSM; O(1) `Heap::open`	SQL insert flat ~17–28 µs/row past 2,000 pages (was `HeapFull` death at ~876 pages / ~145k rows)
CRUD Phase A — write path (2026-07-10)	coalesced index maintenance (`insert_many`, one leaf image per statement); selectivity-gated index-driven UPDATE/DELETE	index WAL 8,868 → 619 B/row; UPDATE-bulk 0.11× → 0.34× vs PG
CRUD Phase B — read path (2026-07-10)	`count_visible` header-only counting; `deform_row` partial decode (stop after last needed attr)	`SELECT COUNT(*)` **2.81× faster than Postgres**; dec/row 2.00 → 0.00 on filtered SELECT
Milestone P — parallel scan (default-on, item 15)	pages partitioned across `std::thread::scope` workers over the shared mmap; partial COUNT aggregate; parallel index-candidate resolution; global worker budget + deadline checks	COUNT 3.82×, filtered COUNT 6.6×, filtered SELECT 6.41× (PG scan lead +540% → +82%); scan 5.6M → 35.7M rec/s @1M rows

Durable IVF-Flat (P3.c)	posting lists = DiskBTree; centroids WAL-logged	recall@10 = 1.000; O(1) open (in-RAM HNSW: 30 s build for 1,200 vectors → 24 ms)
Vacuum + autovacuum (M10, A1–A4)	reader-aware horizon, index scrub before slot reuse, threshold-triggered background pass	churned point reads 35 → 5.85 µs; heap bounded ~35 vs ~82 pages
Auto-checkpoint (P1.e)	time/WAL-size triggers at quiescent points	WAL bounded ~50–154 KB vs 1.17 MB unbounded, flat throughput

11.3 The multi-model thesis, quantified

Measurement	Result
W0→W4 decomposition ladder (add one model per rung, one durable commit)	W4/W0 = **1.12–1.20×** at 1k–10k rows — four models cost ~one, because they share one fsync
vs replaced stack (PG row + pgvector + graph table + outbox; 4 commits), real F_FULLFSYNC durability	**3.61× faster** (250 vs 69 txns/s); ~parity under cheap VM fsync (win is durability-cost-dependent — stated honestly)
Crash consistency across the four writes	**Unconditional win:** 0 orphans vs a torn record (crash-harness `item16_*` proofs)
Bulk scale (2026-07-13 report)	bulk insert ~31k rec/s flat to 2M rows; full-heap scan ~17.5M rec/s at 1–2M rows; engine handles ≥10M
Where unidb wins single-model	COUNT(*) 2.81× vs PG; graph adjacency ~parity; `poll_events` 20.8–983.7 µs vs PG SKIP-LOCKED ~2.6–3.1 ms; embedded point reads ~5× vs PG
Where it still loses (tracked)	UPDATE bulk 0.34× vs PG (needs HOT-style updates — item A2, deferred); SSE subscriber scaling; `poll_events` linear in table size

> **Measurement hygiene rules (standing):** one bench process at a time; trust absolute numbers + internal counters (dec/row, cols/row, WAL B/row) over noisy single-run ÷PG ratios; verify which executor route and which toggles a workload actually exercises before trusting a number (the “no parallel win” report was a default-off toggle, not a missing win); durability lenses must be matched (macOS `open_datasync` made PG look 40× faster — an illusion).

12. Correctness strategy & locked decisions

12.1 Test inventory

374 library unit tests + full integration suites (concurrency matrix, per-domain MVCC/abort-visibility suites, graph locking, queue vacuum, server tests, attach-client tests) + **31 crash-harness tests** + a recovery property test under both durability policies + a `loom` model of the crabbing latch order + a 28-cell production-shaped concurrency correctness matrix (pass/fail oracles: exact visible-id sets, no duplicate ids in any snapshot, COUNT agreement, repeatable re-reads, sum invariance, index-vs-scan agreement — with CPU-contention spinners and hang deadlines). Gates on every PR: `cargo fmt`, `clippy -D warnings`, all tests + crash harness green, benchmarks recorded.

12.2 Locked decisions D1–D13

#	Decision	Where enforced / status
D1	Steal + no-force (ARIES): redo **and** undo logging	`wal.rs`; fulfilled by group-committed force-log-at-commit (2026-07-09)
D2	Per-statement mini-transaction is the atomic unit	`wal.rs` bracketing unchanged; only fsync timing moved to commit
D3	Control file is the recovery root	`control.rs`; extended with `catalog_root`, `next_xid` (signed off)
D4	Tuple header reserves MVCC bytes up front	`page.rs` 24-byte header; in use since M1
D5	No dirty page flushes ahead of durable WAL	`flush_page()` + `find_victim()`; debug asserts; eviction-forced sync; crash harness
D6	Single-file data store (WAL separate)	unchanged for data; WAL segmented in P6.a (sign-off recorded)
D7	Crash-injection harness, simple by design	`tests/crash`, 31 tests; grows with every durability mechanism
D8	8 KiB pages, init-time config, immutable after	`format.rs`; baked into control file
D9	Little-endian, CRC32+LSN per page, magic+version	`FORMAT_VERSION = 5`
D10	RC default; RR available on same snapshots	`txn.rs` snapshot lifetime; + SERIALIZABLE (P1.d)
D11	`on_read` / `on_write` seam; SSI is an addition	seam in `concurrency_hooks.rs`; SSI at the executor
D12	SI abort-on-conflict first; RC re-evaluation	RC re-reads via fresh snapshot; RR/SER `SerializationFailure`; SSI pivot abort
D13	Structured logging from day one	`tracing` on WAL/checkpoint/recovery; `/metrics`, `/stats`, audit log

Changing any locked decision requires explicit human sign-off recorded in `PROGRESS.md`. Corrections to shipped design are made as inline correction notes, never silent rewrites.

13. Known limitations & tech debt (honest registry)

Area	Limitation	Status
SQL surface	No window functions, recursive CTEs, FULL OUTER/NATURAL joins (`USING` shipped 2026-07-13), views, triggers, stored procedures; single-column indexes only; first-indexable-term-wins in the fast path	OPEN (Phase-4 documented limits)

DDL	Catalog not MVCC-versioned — `CREATE TABLE` in an aborting txn is not rolled back through crash recovery (request-level rollback exists)	**OPEN** (tracked)
SSI	Row-granularity only — no predicate locks, so no phantom protection; write-skew pair may both abort (sound, over-conservative)	**OPEN** (reduced form as planned)
Write path	UPDATE pays insert-new-version cost (~619 B/row WAL) where Postgres uses HOT (in-place, no index touch) — the path to UPDATE parity (item A2)	**DEFERRED** (ROI vs charter)
Read path	Only COUNT(*) is pushed into parallel workers (SUM/GROUP BY partial aggregates, LIMIT early-stop filed); no visibility map / index-only scans; no planner column pruning in `query_exec`	**FILED**
Queue	`poll_events` full-scans `__events__` (needs a `seq` index); `__consumers__` ack versions never vacuumed; SSE is poll-per-subscriber (162.6 ms at 50 subscribers)	**OPEN**
Vacuum	Global (not per-table) accounting; no cost-based throttle; no catalog-page or cross-page/ `VACUUM FULL` reclamation; catalog blob strands a page per DDL change	**OPEN**
Catalog	Single ~8 KiB page blob — a very wide analyzed schema can overflow it (multi-page catalog filed)	**OPEN**
Concurrency	Graph/LOB writes serialize on `write_serial` ; B-Tree crabbing is exclusive-latch (Lehman-Yao B-link would overlap same-subtree descents — format-bump-gated)	**FILED**
Replication	PITR by LSN only (no commit timestamps); replica pages allocated after the base need re-basing; no logical replication	**OPEN**
Security	Verify-only JWT (server never issues tokens); encryption-at-rest deferred (D9-gated, fights the mmap store)	**DEFERRED**
Graph	Cypher single-hop read-only; nodes opaque i64; no reverse (`to_id`) traversal; CSR retired (needs a generation marker to ever return)	**OPEN**
Maturity	Rigorous and crash-tested, but early-stage — not battle-hardened against Postgres's ~30 years	**HONEST CAVEAT**

14. Future scope — Postgres & Supabase alignment for production readiness

This section maps the road from "strong prototype → early product" to a production-grade system, aligned against the two reference points users actually compare unidb to: **Postgres** (engine-to-engine) and **Supabase** (engine + platform). Framing per the project charter: unidb is *"SQLite for AI-native, multi-model apps"* — the goal is not to clone either, but to close the gaps that block production adoption while protecting the moat (multi-model atomicity). Items marked `NON-GOAL` are explicitly out of charter and listed so the boundary is documented.

14.1 Postgres-alignment track (engine completeness)

Tier P0 — correctness & transactional completeness (do first)

Item	What / why
Transactional DDL	MVCC-version the catalog (or WAL-logged catalog undo) so `CREATE/ALTER/DROP` roll back with the transaction and through crash recovery — the narrowest real correctness gap today. Also unblocks session-scoped DDL over REST (currently rejected as `DDL_IN_SESSION`).
Savepoints / nested rollback	`SAVEPOINT` / `ROLLBACK TO` — the undo-action list is already per-transaction and ordered, so this is a marker into `Vec<UndoAction>` plus lock bookkeeping. Postgres-without-savepoints session semantics is a documented papercut.
Full SSI (predicate locks)	Phantom protection via predicate/range locks (or Postgres-style SIREAD on index ranges) — upgrades SERIALIZABLE from the reduced row-granularity form. The D11 seam was built for exactly this retrofit.
Lock wait-then-re-evaluate at RC	Block on a still-active writer then EvalPlanQual-re-check (the wait queue exists since Phase 5) instead of the no-wait `WriteConflict` — removes the biggest app-visible retry burden.
Referential integrity, complete	FK enforcement of row existence + `ON DELETE/UPDATE CASCADE/RESTRICT/SET NULL` (today FK checks referenced-table existence only). UNIQUE via a unique-index path instead of a synchronous heap scan once index currency can be proven.

Tier P1 — SQL surface parity (what real apps hit next)

Item	What / why
Views + materialized views	Plain views are a planner rewrite (the RLS mechanism generalizes); materialized views can reuse the durable-index build path + refresh machinery.
Window functions	The most-requested Phase-4 gap; builds on the existing sort/aggregate operators (partition = sort + frame walk).
Triggers & stored procedures	Row-level triggers first (the event-capture hook `send_event_capture` is already an inline row-write hook — generalize it); a full procedure language is a later, larger decision.
Recursive CTEs + FULL OUTER/NATURAL	Completes the Phase-4 grammar (`USING` already shipped 2026-07-13; `NATURAL` desugars to it once `FULL OUTER`'s `CASE` / `COALESCE` blocker lands); recursive CTEs also unlock multi-hop graph traversal through SQL.
Richer types	Arrays, ENUM, INTERVAL, NUMRANGE, collations — additive row-encoding tags (the D4 pattern), no format bump.
Index power	Multi-column + expression + partial indexes; cost-based index *selection* (today: first indexable term wins); index-only scans backed by a visibility map (also the true COUNT accelerator at scale).
`CREATE POLICY` SQL	Surface the existing RLS mechanism as standard SQL DDL with per-role policies.

Tier P2 — performance parity work (measured gaps, filed)

Item	What / why
HOT-style updates (item A2)	Forward-chained / RowId-preserving in-page updates that skip index maintenance when indexed columns are unchanged — the path from UPDATE 0.34× to parity. Fiddly against insert-new-version MVCC; needs its own design review.
Parallel partial aggregates beyond COUNT	SUM/AVG/MIN/MAX/GROUP-BY pushed into scan workers; `ORDER BY … LIMIT` early-stop + streaming.
Background WAL writer	Smooths eviction-forced syncs under memory pressure (noted, not required for correctness).
B-link (Lehman-Yao) tree	Right-linked internal nodes let same-subtree descents overlap — the next concurrency step after crabbing (format-bump-gated).
Queue scalability	`seq` index for `poll_events` (flat instead of linear), `__consumers__` vacuum, and a wake primitive so SSE pushes instead of polling per subscriber.
Vacuum maturity	Per-table accounting + `vacuum_table`, cost-based throttle, catalog-page reclamation, `VACUUM FULL`-style cross-page compaction, IVF re-training as a maintenance op.

Tier P3 — operational parity (production operations)

Item	What / why
Wire protocol / drivers	A pgwire-compatible listener would inherit the entire Postgres driver ecosystem (psql, ORMs, BI tools) — the single highest-leverage ecosystem move; alternatively first-class native drivers beyond Rust.
Time-based PITR	Commit timestamps in the WAL commit record → `restore(…, target_time)`; today PITR is by LSN only.
Logical replication / CDC	The event queue is already a change feed with offsets — formalize it into table-level logical replication and external CDC connectors.
Encryption at rest	Deferred D9-gated follow-up; page-level encryption must be reconciled with the mmap-as-storage design (likely a buffered page-cache mode).
Connection/session model	Connection pooling semantics, per-session GUC-style settings, statement-level metrics; richer `/stats` (per-table, per-index, per-query).
Upgrade & migration story	`FORMAT_VERSION` migration paths (none have shipped externally yet — cheap now, expensive later), online schema-change guidance, config file story beyond env vars.

14.2 Supabase-alignment track (platform surface)

Asymmetric by design: this compares unidb + its small REST server against Postgres + a whole product layer. The near-term goal is “production-credible backend-in-a-box”, not platform parity. **Update 2026-07-13:** five of these tracks were filed as backlog items 20–24, and **three shipped the same day** — events/realtime dispatcher (20, Milestone 20), observability metrics (21), and the logs surface (22). Storage service (23) and authz v2 policies (24) remain filed follow-ups. The rows below are marked accordingly.

Capability	Today	Future scope
Auth platform	Verify-only JWT (HS256); users/roles/GRANT; JWT `sub` → user	A GoTrue-shaped auth service (token issuance, signup/login, refresh, OAuth providers, magic links, MFA) as a separate feature-gated service in front of the existing verify-only core; JWKS/RS256 verification so third-party IdPs plug in. Row-level policy engine v2 is filed as item 24. HIGH VALUE

Client SDKs	REST + Rust attach client	TypeScript/JS SDK first (the Supabase-DX core), then Python; generated from the REST contract so they track the server; session (`X-Txn-Id`) support in SDKs. Embedded bindings (PyO3, napi-rs) parked until the engine is production-solid.
Auto API richness	Hand-built REST routes; SQL over POST	PostgREST-style filtering/ordering/embedding on `/rows` (composable query params), OpenAPI schema generation from the catalog, optional GraphQL façade.
Realtime	**Shipped (M20, 2026-07-13):** ephemeral SSE resume (`Last-Event-ID` / `?from_seq=` / `?table=`) + `unidb-dispatch` fan-out (webhook→dead-letter, broadcast rooms) with proven at-least-once + zero-loss-across-crash delivery on the durable offsets — the moat vs Supabase Realtime (not durable by default). SHIPPED	Remaining: a commit-time wake primitive (kill the poll loop) + an engine-side `seq` index for large backlogs, RLS-filtered payloads, and a WebSocket transport. MOAT-ADJACENT
Observability	**Shipped (item 21, 2026-07-13):** lock-free per-chokepoint metrics (statement/fsync/buffer-pool/lock-wait/horizon-age/parallel-worker/session) over `stats()` / `/stats` / `/metrics` , <1% overhead. SHIPPED	Per-table (not engine-global) dead-tuple estimates; exact percentiles instead of log-bucket estimates.
Logs	**Shipped (item 22, 2026-07-13):** JSON structured logs, `request_id` / `txn_id` correlation across app/slow-query/audit logs, bounded cursor-paged `GET /logs` ; documented CloudWatch/Datadog/Loki shipping contract. SHIPPED	A studio Logs tab (out of repo); live tail over SSE (reuses M20 framing).
Storage	In-engine chunked/streamed LOBs (multi-GB, transactional)	Streaming REST routes for LOBs; optional S3 tiering for cold objects (item 23, filed; parked behind local-disk economics; reverses D6 — needs sign-off). Transactional LOBs are already a differentiator vs S3-backed storage.
Dashboard / Studio	—	A minimal web console over the existing REST surface: schema browser (`GET /tables`), SQL editor with `EXPLAIN` , stats/metrics panels, RLS/role editor. Read-only first.
Migrations tooling	SQL DDL + docs	Versioned migration files + CLI (`unidb migrate`), diff-based schema generation once transactional DDL (P0) lands.
Edge functions / hosting	—	NON-GOAL — no cloud control plane by charter; document integration patterns (run embedded inside the function runtime instead — a story Supabase cannot offer).

14.3 Scale-out (parked, explicitly)

- **Columnar / HTAP** — gated on real analytics demand; opposite axis to the multi-model thesis.
- **Distributed / sharded writes** — reverses the single-primary charter and strains cross-model atomicity; a separate multi-year project. The confirmed target remains **a strong single node + read replicas** (100s of GB, high read throughput).
- **S3 / tiered storage** — relevant past local-disk economics; behind replication maturity.

14.4 Suggested sequencing (ROI-ordered, to be re-derived per session)

1. **Transactional DDL + savepoints** (P0) — correctness first, unblocks migrations tooling and session DDL.
2. **TypeScript SDK + PostgREST-style filtering + push-based realtime** — the shortest path to “Supabase-shaped DX” on the existing moat.
3. **Views + window functions + index power** (P1) — the SQL gaps real apps hit first.
4. **pgwire listener** (P3) — the ecosystem multiplier; evaluate cost honestly before committing.
5. **HOT updates + queue indexes** (P2) — close the two worst measured single-model gaps.
6. **Auth service + dashboard** — platform credibility for production adoption.

> **Standing rule for all of the above** (charter §0.6): every item gets an expert design review before implementation, an adversarial stress test + honest baseline benchmark when it ships, gated optimizations (never unconditional), and its ROI re-derived at execution time — the ordering above is a proposal, not a commitment. No locked decision (D1–D13) may be reopened without recorded human sign-off.

---

unidb Engine Architecture & Design Reference · generated 2026-07-13 from the repository’s canonical docs (`engine_design.md` plus the 2026-07-13 merges: Milestone 18 introspection, `JOIN … USING`, Milestone 20 realtime dispatcher, and items 21 observability / 22 logs). This PDF is a distilled snapshot; when it disagrees with `CLAUDE.md` / `PROGRESS.md` / `MEMORY.md`, those win. Regenerate: edit `docs/design/unidb_engine_architecture.html` and print to PDF (see `docs/design/design_index.md`).